Breaking the Byte Barrier: Why 16-Bit Native Entropy Coding Matters for Medical Image Compression

Kuldeep Singh
Innovaccer Inc.
kuldeep.singh@innovaccer.com

Code: https://github.com/pappuks/medical-image-codec

Abstract

The dominant entropy coders in modern compression systems—Huffman, arithmetic coding, LZ77, and FSE/ANS as used in Zstandard—were designed for 8-bit byte streams. Medical imaging breaks this assumption: DICOM stores pixels at 10–16 bits per sample, yielding symbol alphabets of 1,024 to 65,536 entries. We show that this mismatch is not merely inconvenient but structurally costly: byte-splitting a 16-bit residual distribution discards mutual information between the high and low bytes, inflating compressed output by 10–22% compared to a 16-bit-native entropy coder.

We present **MIC** (Medical Image Codec), a lossless codec that addresses this gap with four contributions: (1) a 16-bit-native FSE implementation supporting up to 65,535 symbols, outperforming Delta+Zstandard by 10–22% on all tested modalities; (2) a four-state interleaved ANS decoder targeting instruction-level parallelism, with a formal latency model showing 66–142% speedup and zero compression ratio penalty; (3) empirical evidence for the *simplicity thesis*—that a 500-line Delta+RLE+FSE pipeline matches or beats JPEG 2000's wavelet+EBCOT on 7/8 medical modalities at dramatically higher throughput; and (4) browser-native decoding via a 20 KB pure JavaScript ES module achieving 483 MB/s using parallel strips.

We evaluate MIC on 21 clinical DICOM datasets spanning 10 modalities using fair in-process benchmarking against HTJ2K (OpenJPH), JPEG-LS (CharLS), and Delta+Zstandard. MIC achieves compression ratios of 1.7×–8.9× with decompression throughput up to 16 GB/s on 64-core ARM64 (geometric mean ≈7.5 GB/s). MIC-4state-C exceeds HTJ2K decompression speed on 19/21 images single-threaded (ARM64) and 16/21 (Intel AMD64, AVX2). A five-level 5/3 wavelet alternative matches or exceeds HTJ2K's compression ratio on 7/8 original modalities and exceeds HTJ2K decompression speed on 4/8 (large images with high spatial correlation: CT, CR, XR, MG3). PICS-C-8 (8 parallel strips) exceeds HTJ2K on all 21 images. The RGB pipeline (YCoCg-R + Delta+RLE+FSE) achieves 1.56×–6.24× lossless compression on ultrasound and visible light images from the NEMA compsamples dataset.

1 Introduction: The 16-Bit Alphabet Gap

Medical imaging generates enormous volumes of data. A single digital breast tomosynthesis (DBT) study produces 69 or more frames of 2457×1890 pixels at 10-bit depth, totalling over 600 MB of raw pixel data. Whole slide pathology images routinely exceed 100,000×100,000 pixels. Efficient lossless compression is essential for archival storage, network transmission, and real-time rendering in clinical PACS and diagnostic viewers.

The DICOM standard [1] supports several lossless transfer syntaxes, including JPEG 2000 [2], JPEG-LS [4], and RLE Lossless. Each has well-documented tradeoffs between compression ratio and decompression speed [26, 27]. But all share a less-discussed limitation: they were designed in an era of 8-bit data.

JPEG's Huffman and arithmetic coders, JPEG-LS's Golomb-Rice coder, JPEG 2000's MQ coder, and Zstandard's FSE backend all treat each coding unit as a small fixed-size symbol (typically 4–8 bits). FSE as used in Zstandard [10] caps its alphabet at 4,096 symbols. Deflate, LZ4, Brotli, and JPEG XL [6] operate on 8-bit byte streams. This is not merely historical—it is a design constraint that propagates through table sizes, memory layouts, and optimization strategies.

Medical images break this assumption. DICOM stores pixels at 10, 12, or 16 bits per sample, yielding alphabet sizes of 1,024, 4,096, or 65,536 symbols. Even after spatial delta prediction, the residual alphabet

for a 16-bit CT image can span tens of thousands of distinct values. An entropy coder that assumes an 8-bit alphabet must either re-encode 16-bit symbols as byte pairs (losing inter-byte correlation) or truncate the alphabet at 256 (discarding rare symbols).

This mismatch has measurable consequences. We show in Section 9 that applying Zstandard to delta-encoded 16-bit medical images loses 10–22% of achievable compression compared to a 16-bit-native approach. The loss is structural: a run of 1,000 identical 16-bit zero residuals is a single same-run record in a 16-bit RLE, but 2,000 bytes with no obvious structure to a byte-level LZ77 matcher.

1.1 Contributions

This paper makes four contributions:

1. We expose and quantify the **16-bit alphabet gap** and demonstrate a 16-bit-native RLE+FSE pipeline that closes it (Section 3).
2. We formalize **multi-state ANS decoding as an ILP design axis**, with a latency model, correctness argument, and platform-specific assembly achieving 66–142% speedup with no ratio penalty (Section 4).
3. We present the **simplicity thesis**—empirical evidence that a simple spatial predictor paired with 16-bit entropy coding outperforms or matches wavelet+context-adaptive approaches at dramatically higher throughput, at ∼40× lower implementation complexity than HTJ2K (Section 5).
4. We demonstrate **browser-native decoding as a distribution primitive**—a 20 KB JavaScript decoder and PICS strip parallelism achieving 483 MB/s in a browser (Section 11.4).

We evaluate against HTJ2K, JPEG-LS, and Zstandard using fair in-process benchmarking (Sections 7–9), correcting earlier subprocess-based measurements and discussing the benchmarking methodology pitfall (Section 6).

2 Background and Related Work

2.1 Lossless Image Compression in DICOM

JPEG 2000 [2] employs a 5/3 reversible integer wavelet transform with embedded block coding (EBCOT). It achieves excellent compression ratios but its bit-plane coding and context modeling impose significant computational overhead during decompression. **HTJ2K** [3] replaces EBCOT with a faster block coder, substantially improving decode speed while maintaining JPEG 2000 compatibility.

JPEG-LS [4] uses context-adaptive prediction (MED predictor) followed by Golomb-Rice coding. It provides fast encoding and decoding with moderate compression ratios. JPEG-LS is well-suited to smooth images but its fixed prediction model cannot adapt to the varying statistics of different medical modalities.

RLE Lossless (DICOM Transfer Syntax 1.2.840.10008.1.2.5) uses byte-level PackBits run-length encoding. It achieves poor compression ratios (typically $< 1.5\times$) because it operates on bytes rather than 16-bit pixel values and lacks any decorrelation stage.

2.2 Entropy Coding and the Byte Assumption

Traditional arithmetic coding achieves near-optimal compression but is inherently sequential. Mahapatra and Singh [12] addressed this with a parallel-pipelined FPGA implementation, demonstrating that hardware acceleration can overcome the throughput limitations of sequential arithmetic coders.

Asymmetric numeral systems (ANS) [7, 8] generalize arithmetic coding into a single-state machine that replaces interval subdivision with integer state transitions. Finite State Entropy (FSE) [9] is a table-driven tANS implementation achieving near-optimal compression with O(1) per-symbol operations. FSE was popularized by Zstandard [10]. Recent theoretical analyses have formalized ANS efficiency bounds [13] and provided statistical interpretations of the coding process [14].

All of these coders share one assumption: small alphabets. Huffman coding becomes impractical above ∼4,096 symbols. FSE in Zstandard caps at 4,096. The entire ecosystem was built for bytes.

2.3 The Information-Theoretic Cost of Byte-Splitting

Consider a 16-bit symbol X decomposed into its high byte X_H and low byte X_L. The joint entropy satisfies:

$$H(X) = H(X_H, X_L) = H(X_H) + H(X_L \mid X_H). \quad (1)$$

A byte-level coder that processes X_H and X_L independently achieves at best:

$$H(X_H) + H(X_L) = H(X) + I(X_H; X_L), \quad (2)$$

where $I(X_H; X_L)$ is the mutual information between the two bytes. This mutual information is *discarded*—it represents correlation that the byte-level coder cannot exploit.

For medical image residuals after delta prediction, this mutual information is substantial. When a residual is small (say, ±30), the high byte is always `0x00` (or `0xFF` for negative values stored as uint16), and the low byte carries all the information. The high byte is nearly deterministic given the low byte—yet a byte-level coder must still encode it.

Quantification: On our test images, $I(X_H; X_L)$ of delta residuals ranges from 0.3 bits/symbol (MR) to 1.1 bits/symbol (MG3). This corresponds to 8–22% of the total entropy—closely matching the 10–22% empirical advantage of MIC over Delta+Zstandard (Table 13). The gap is causal, not incidental.

2.4 Signal Processing Approaches for Lossless Compression

Image compression research has drawn extensively on classical signal processing [21–24]. For *lossy* compression, wavelet decomposition is highly effective—high-frequency subbands can be quantized with little perceptual impact.

For *lossless* compression, the picture is more nuanced. Four constraints erode the advantages of the filter-bank approach:

1. **Coefficient overflow.** Even with integer lifting (the 5/3 Le Gall wavelet), the predict step expands coefficients by up to $(3/2)^5 \approx 7.6\times$ the input range for 5 levels. This forces promotion to int32 or escape coding, inflating the compressed stream.

2. **The update step reintroduces correlation.** For images already well-predicted by a simple linear filter, the wavelet update provides no benefit to the smooth regions that dominate medical images.

3. **Higher memory bandwidth.** The 2D wavelet inverse transform requires two sequential passes at int32 width (4 bytes/sample vs. 2 bytes/sample for delta).

4. **Entropy coding mismatch.** Wavelet subbands have distinct statistical characteristics requiring subband-specific entropy models (e.g., EBCOT in JPEG 2000).

By contrast, simple spatial prediction leaves a single residual image whose statistics are *homogeneous*—tightly clustered around zero everywhere, making the residual ideal for a uniform 16-bit entropy scheme applied once.

2.5 Color Transforms for Pathology Imaging

The YCoCg-R transform [25] is a reversible integer approximation of RGB-to-YCbCr. It decorrelates RGB channels into luminance (Y) and chrominance (Co, Cg) with no loss of precision. MIC uses YCoCg-R for whole slide image and single-frame RGB compression.

3 Pipeline Design: Delta + 16-Bit RLE + Extended FSE

MIC processes 16-bit greyscale pixels through three sequential stages:

```
Raw 16-bit Pixels
↓ pixel − avg(top, left)
Delta Encoding
↓ same / diff runs
Run-Length Encoding
↓ table-driven ANS
FSE (tANS) Entropy Coding
↓
Compressed Bitstream
```

3.1 Delta Encoding

The delta encoder computes the prediction residual for each pixel as:

$$r_{i,j} = x_{i,j} - \left\lfloor \frac{x_{i-1,j} + x_{i,j-1}}{2} \right\rfloor \tag{3}$$

where $x_{i,j}$ is the pixel at row i, column j. Boundary pixels use only the available neighbor; the top-left corner pixel is stored raw.

For 16-bit images, residuals can span $[-65535, +65535]$. Rather than widening to 32 bits, MIC uses an **overflow delimiter** scheme derived from the effective bit depth $d = \lceil \log_2(\max(x) + 1) \rceil$: residuals with $|r| \leq 2^{d-1} - 1$ are stored directly as uint16; larger residuals are preceded by delimiter value $2^d - 1$, followed by the raw pixel value.

Why not the MED predictor? The JPEG-LS MED predictor was tested through the full RLE+FSE pipeline. Result: geometric mean improvement of only +0.9% on ratio, with MG4 regressing by −1.7%. MED decompression is 1.5–2× slower due to the three-way conditional branch and diagonal neighbor dependency that prevents the branch-free interior loop optimization. The average predictor is retained: the compression gains are small and inconsistent while the speed penalty is significant.

3.2 Run-Length Encoding (16-Bit Native)

After delta coding, medical images produce long runs of identical residual values. MIC's RLE stage encodes the symbol stream as alternating **same runs** (a count followed by a single repeated value) and **diff runs** (a count followed by distinct values).

A key design constraint is that the minimum run length is 3, which guarantees that RLE output is **never larger than the input**: a same-run header costs 2 symbols encoding 3 input symbols (net -1), and a diff-run header costs 1 symbol with $n \geq 3$ values verbatim (net $+1$), but diff runs can only follow same runs. This eliminates the need for expansion detection or fallback paths.

Why 16-bit RLE matters: A run of 1,000 identical 16-bit zero residuals is a single same-run record in MIC's RLE (2 uint16 words). The same data viewed as 2,000 bytes has no obvious run structure to a byte-level compressor—the zero bytes are interleaved with whatever byte pattern represents zero in little-endian. Word-level structure is invisible at the byte level.

Run headers use a midpoint protocol: counts $\leq$ `midCount` signal same runs; counts $>$ `midCount` signal diff runs.

3.3 FSE Entropy Coding (Extended to 65,535 Symbols)

The final stage encodes the RLE output using table-based ANS (tANS). **MIC extends FSE to support up to 65,535 distinct symbols** (versus the 4,095 limit in Zstandard), which is necessary for 16-bit medical images where delta residuals can span a wide range.

Encoding processes symbols in reverse order, building the compressed bitstream from the end. For each symbol the encoder transitions between states using a pre-computed symbol transition table (`symbolTT`).

Decoding reads bits forward using a decode table (`decTable`) indexed by current state. Each decode step requires only a table lookup, a bit read, and an addition—no divisions or multiplications.

Adaptive table sizing: MIC automatically adjusts the FSE table log (default 11; bumped to 12 when symbol density $>$ 128 distinct symbols with $>$ 32 data points each; bumped to 13 when `symbolLen` $>$ 512 and density $>$ 16). This provides 4–7% better compression ratios on CR and mammography images (Table 1).

Dynamic table sizing: Encode and decode tables are sized to the actual symbol range rather than the theoretical 65,536 maximum. For 8-bit images, this reduces the working set from 512 KB to approximately 2 KB, substantially improving cache utilization.

Table 1: Effect of adaptive tableLog (11→12).

Image	tableLog=11	Adaptive	Gain
CR	3.47×	3.63×	+4.4%
MG1	7.99×	8.57×	+7.1%
MG2	7.98×	8.55×	+7.1%

Canonical Huffman alternative: MIC also supports a canonical Huffman entropy backend [11]. It typically produces 1–3% smaller output than FSE but decodes more slowly due to variable-length code lookup.

4 Multi-State ANS: ILP as a First-Class Design Axis

4.1 The Serial Dependency Problem

The standard FSE decode loop has a serial carried-dependency chain:

$$\texttt{state}[n{+}1] = \texttt{decTable}[\texttt{state}[n]].\texttt{newState} + \texttt{getBits}(\texttt{decTable}[\texttt{state}[n]].\texttt{nbBits}). \quad (4)$$

With a table-lookup latency of approximately $L \approx 4$ cycles (L1 cache hit), the loop is limited to roughly one symbol per L cycles regardless of available execution units.

Modern CPUs have 4–6 execution ports. A single-state ANS decoder uses exactly **one** dependency chain, leaving 75–83% of the hardware capacity idle. This gap between algorithmic throughput (1 symbol/L cycles) and hardware capacity ($\sim$4 symbols/L cycles) is a 4× inefficiency that no amount of microarchitectural optimization can close within the single-state framework.

4.2 Multi-State Interleaved Decoding

MIC breaks this dependency by running multiple independent state machines on interleaved symbol positions. The four-state decoder uses four chains (A, B, C, D) cycling over positions modulo 4:

$$\begin{array}{lll} \text{symbol}[0] \leftarrow \text{stateA}, & \text{symbol}[4] \leftarrow \text{stateA}, & \ldots \\ \text{symbol}[1] \leftarrow \text{stateB}, & \text{symbol}[5] \leftarrow \text{stateB}, & \ldots \\ \text{symbol}[2] \leftarrow \text{stateC}, & \text{symbol}[6] \leftarrow \text{stateC}, & \ldots \\ \text{symbol}[3] \leftarrow \text{stateD}, & \text{symbol}[7] \leftarrow \text{stateD}, & \ldots \end{array}$$

The four chains are completely independent: each has its own state variable, table lookups, and bit-extraction operations. The CPU's out-of-order engine sees four simultaneous table-lookup address streams.

Correctness: The encoder writes symbols at positions $0, 4, 8, \ldots$ into state A's bitstream; positions $1, 5, 9, \ldots$ into state B's; etc. Since each chain operates on a strict subset of positions with no data dependency on other chains, correctness follows from the correctness of single-state FSE applied independently to each subset.

Latency model: Let L be the table-lookup latency. The k-state decoder processes k symbols per L cycles (amortized):

Table 2: Multi-state latency model vs. empirical speedup.

States (k)	Amortized latency	Theoretical	Empirical
1	L cyc/sym	1.0×	1.0×
2	$L/2$ cyc/sym	2.0×	1.3×
4	$L/4$ cyc/sym	4.0×	2.0×

The empirical speedup is below theoretical maximum due to: (1) bit-reader refill overhead, (2) instruction cache pressure from the unrolled loop, and (3) memory bandwidth limits on the compressed stream reads.

Stream format: The compressed stream is prefixed with a magic sequence (`0xFF`, `0x02` for two-state; `0xFF`, `0x04` for four-state) followed by a 4-byte symbol count, enabling the auto-detect dispatcher to route streams transparently. The magic byte `0xFF` cannot appear as a valid single-state FSE header, so all three formats coexist without ambiguity.

4.3 Platform-Specific Assembly Kernels

AMD64 (BMI2): The kernel uses `SHLXQ`/`SHRXQ` for 3-operand variable shifts, enabling four independent bit-extraction sequences per iteration without CL register contention. Standard x86 shifts require the shift count in the CL register, serializing multi-symbol extraction.

ARM64: The kernel uses `LSLV`/`LSRV` variable-shift instructions which natively take the count from any general-purpose register. The kernel still benefits from four-chain ILP because the table-lookup latency is the binding constraint.

Fallback: A pure-Go four-state loop provides identical functionality on all other platforms (WebAssembly, RISC-V, etc.).

4.4 FSE Decompression Throughput

Table 3 shows isolated FSE decompression throughput across state counts. The largest gains occur on mammography where the large payload keeps the decode table hot in cache and the assembly loop saturates the execution units.

Table 3: Isolated FSE decompression throughput: 1-state vs. 2-state vs. 4-state (Intel Xeon @ 2.80 GHz, MB/s over uncompressed RLE symbol stream).

Image	1-state	2-state	4-state	4 vs. 1
MR	164	207	298	+82%
CT	113	126	195	+73%
CR	177	182	310	+75%
XR	193	172	320	+66%
MG1	158	207	380	+140%
MG3	576	890	1,343	+133%
MG4	256	321	620	+142%

4.5 Broader Significance

This multi-state approach demonstrates that **entropy coder design should target ILP width, not just coding efficiency**. The traditional compression research agenda focuses on minimizing bits per symbol. But on modern hardware, the binding constraint is often symbols per cycle. A coder that is 1% less efficient but 3× faster at decoding is overwhelmingly preferable for read-heavy workloads.

This connects to a broader trend: Giesen's interleaved entropy coders [15, 16] demonstrated the value of multi-stream rANS; Lin et al. [17] extended parallel rANS to decoder-adaptive scalability; Weissenberger and Schmidt [18] showed massively parallel ANS decoding on GPUs. MIC's multi-state FSE is the pure-software, single-core expression of this principle—targeting ILP rather than thread-level parallelism.

5 The Simplicity Thesis: When Wavelets Lose to Delta

5.1 Proposition

For images where a simple spatial predictor achieves >90% of optimal decorrelation (residuals are unimodal around zero with low kurtosis), additional transform complexity yields diminishing returns on lossless ratio while incurring proportional throughput cost.

Medical images (except noisy X-ray) fall squarely in this "predictor-sufficient" regime. The wavelet wins

by only 1–5% on ratio but imposes 2× memory bandwidth. The correct complexity budget for lossless medical compression is in the entropy coder (multi-state ILP), not the decorrelator.

5.2 Wavelet Implementation

We implemented the Le Gall 5/3 integer wavelet [21] using the lifting scheme [22, 23]—the same transform used in JPEG 2000 lossless mode—as an alternative to delta encoding. The lifting factorization:

$$d[i] = x[2i+1] - \left\lfloor \frac{x[2i] + x[2i+2]}{2} \right\rfloor \tag{5}$$

$$s[i] = x[2i] + \left\lfloor \frac{d[i-1] + d[i] + 2}{4} \right\rfloor \tag{6}$$

The 2D transform applies 1D transforms to all rows, de-interleaves into the Mallat layout, then applies to all columns. Five levels of decomposition with subband-order coefficient scanning: $\mathrm{LL}_5 \to \mathrm{HL}_5 \to \mathrm{LH}_5 \to \mathrm{HH}_5 \to \ldots \to \mathrm{HH}_1$.

SIMD acceleration: The 2D column pass is cache-unfriendly. We optimize with a **blocked column pass** that processes 8 consecutive columns per iteration, so each 32-byte cache-line load serves all 8 columns (∼8× fewer cache misses). On AMD64, AVX2 kernels (`VPADDD`/`VPSUBD`/`VPSRAD`) vectorize the lifting over 8 columns. Compressed output is **bit-identical** to scalar.

5.3 Compression Ratio Comparison

Table 4 shows compression ratios across all 21 images.

5.4 Coefficient Overflow on Wide-Dynamic-Range Images

When the input uses the full 16-bit dynamic range, the 5/3 lifting predict step expands coefficients by up to $(3/2)^5 \approx 7.6\times$ for $L = 5$ levels. Worst-case coefficients require promotion to int32 and escape coding of out-of-range values—inflating the compressed stream. The delta pipeline avoids this issue entirely.

5.5 Decompression Speed

Table 5 shows single-threaded decompression speed on Apple M2 Max.

Wavelet+SIMD leads on 16/21 images. HTJ2K is fastest on MR and the two largest mammography images (MG1, MG2) where its decode table stays warm in cache; MG4 and SC1 are small HTJ2K wins. Delta+RLE+FSE is the consistent third across all modalities at one-quarter the implementation complexity.

Table 4: Compression ratio: Delta+RLE+FSE vs. Wavelet V2 SIMD vs. HTJ2K, 21 images. Bold = best per row.

Image	Delta+RLE+FSE	Wavelet V2	HTJ2K
MR	2.35×	**2.38**×	**2.38**×
CT	**2.24**×	1.67×	1.77×
CR	3.69×	**3.81**×	3.77×
XR	**1.74**×	**1.76**×	1.67×
MG1	**8.79**×	8.67×	8.25×
MG2	**8.77**×	8.65×	8.24×
MG3	2.24×	**2.32**×	2.22×
MG4	3.47×	**3.59**×	3.51×
CT1	**2.79**×	2.49×	2.70×
CT2	**3.49**×	2.87×	3.29×
MG-N	2.24×	**2.32**×	2.23×
MR1	2.09×	**2.14**×	2.13×
MR2	3.28×	3.34×	**3.35**×
MR3	3.93×	4.09×	**4.33**×
MR4	4.12×	**4.18**×	4.21×
NM1	5.15×	5.02×	**5.76**×
RG1	**1.70**×	**1.70**×	1.63×
RG2	4.23×	**4.32**×	**4.32**×
RG3	6.08×	6.82×	**6.99**×
SC1	**3.71**×	3.70×	3.85×
XA1	**5.01**×	4.94×	4.88×

5.6 The Tradeoff Matrix

Recommendation: Use Delta+RLE+FSE for production lossless workflows prioritizing simplicity and portability. Use Wavelet V2 SIMD when compression ratio is the priority and native acceleration is available.

6 Fair Benchmarking: Methodology and Comparisons

6.1 The Benchmarking Methodology Pitfall

An earlier version of our HTJ2K comparison used subprocess-based timings (`ojph_compress`/`ojph_expand`), which inflated apparent HTJ2K latency by approximately 6 ms per invocation (subprocess launch + PGM file I/O). This led to an incorrect claim that MIC was 1.3–1.5× faster. The in-process measurements in this paper correct this.

We report this error explicitly because subprocess-based benchmarking inflates apparent la-

Table 5: Decompression speed (MB/s), single-threaded, Apple M2 Max, in-process. Wavelet+SIMD uses blocked column layout on ARM64 (no AVX2). Bold = fastest of the three per row.

Image	Delta+RLE+FSE	Wavelet+SIMD	HTJ2K
MR	144	248	**265**
CT	191	**316**	307
CR	296	**567**	367
XR	308	**627**	334
MG1	482	678	**810**
MG2	479	697	**790**
MG3	308	**422**	338
MG4	417	516	**548**
CT1	239	**425**	362
CT2	238	**481**	375
MG-N	316	**468**	340
MR1	278	**435**	325
MR2	333	**498**	388
MR3	375	**507**	441
MR4	316	**479**	406
NM1	327	**575**	410
RG1	235	**584**	332
RG2	367	**644**	443
RG3	374	**656**	562
SC1	375	388	**401**
XA1	331	**459**	419

Table 6: Delta+RLE+FSE vs. Wavelet V2 tradeoff matrix.

Axis	Delta+RLE+FSE	Wavelet V2
Compression ratio	Baseline	+1–5% (7/8)
Decomp. speed	Faster	Slower
Memory bandwidth	2 B/sample	4 B/sample
Implementation	∼500 LOC	∼2,000 LOC
Wide dyn. range	Excellent	Poor (escape)
Lossy extension	None	Natural

tency via process startup, file I/O, and linker overhead; for small images this overhead can exceed the actual decompression time. **All comparisons in this paper use in-process CGO library calls** for both MIC and the competing codec, measured on the same hardware in the same process.

6.2 Test Dataset

Table 7 describes the 21 clinical DICOM images used for evaluation.

Data provenance: All test images are drawn from publicly available, fully de-identified DICOM datasets: NEMA WG-04 reference dataset, the Clunie Breast Tomosynthesis Case 1 (69-frame DBT), and the NEMA 1997 CD. All images are de-identified per DICOM PS 3.15 Appendix E. No patient consent or ethics board approval is required for this computational study.

Table 7: Test dataset: 21 clinical DICOM images spanning 10 modalities.

Image	Modality	Dimensions	Raw Size
MR	Brain MRI	256×256	0.13 MB
CT	CT scan	512×512	0.50 MB
CR	Computed radiography	2140×1760	7.18 MB
XR	X-ray	2048×2577	10.1 MB
MG1	Mammography	2457×1996	9.35 MB
MG2	Mammography	2457×1996	9.35 MB
MG3	Mammography	4774×3064	27.3 MB
MG4	Mammography	4096×3328	26.0 MB
CT1	CT scan	512×512	0.50 MB
CT2	CT scan	512×512	0.50 MB
MG-N	Mammography	3064×4664	27.3 MB
MR1	Brain MRI	512×512	0.50 MB
MR2	Brain MRI	1024×1024	2.00 MB
MR3	Brain MRI	512×512	0.50 MB
MR4	Brain MRI	512×512	0.50 MB
NM1	Nuclear medicine	256×1024	0.50 MB
RG1	Radiography	1841×1955	6.86 MB
RG2	Radiography	1760×2140	7.18 MB
RG3	Radiography	1760×1760	5.91 MB
SC1	Secondary capture	2048×2487	9.71 MB
XA1	Fluoroscopy (XA)	1024×1024	2.00 MB

6.3 Compression Ratios

Table 8 shows lossless compression ratios for the Delta+RLE+FSE pipeline across all 21 images. Mammography achieves the highest ratios (up to 8.79×) due to large smooth tissue regions producing near-zero RLE runs over thousands of consecutive pixels. CT with full 16-bit dynamic range achieves 2.24×. X-ray and RG1 achieve the lowest (∼1.7×) due to high-frequency noise or uniform backgrounds.

6.4 Decompression Throughput

Table 9 shows decompression throughput across hardware platforms on the original 8 modalities. Key observations: peak throughput of 16.4 GB/s on MG1 (64-core ARM64) approaches DRAM bandwidth limits; throughput scales roughly linearly with core count; ARM64 outperforms x86 at equivalent core counts.

Table 8: Lossless compression ratios across all variants, 21 images. Bold = best per row. MIC and MIC-4state encode identically. PICS improves ratio on large images via strip-local FSE table adaptation but reduces it on small images.

Image	Raw	MIC	Wavelet	PICS-4	PICS-8	HTJ2K	JPEG-LS
MR	0.13 MB	2.35×	2.38×	2.28×	2.21×	2.38×	**2.52×**
CT	0.50 MB	2.24×	1.67×	2.15×	1.96×	1.77×	**2.68×**
CR	7.18 MB	3.69×	3.81×	3.70×	3.71×	3.77×	**3.96×**
XR	10.1 MB	1.74×	**1.76×**	1.75×	**1.76×**	1.67×	**1.76×**
MG1	9.35 MB	8.79×	8.67×	8.84×	8.87×	8.25×	**8.91×**
MG2	9.35 MB	8.77×	8.65×	8.83×	8.85×	8.24×	**8.90×**
MG3	27.3 MB	2.24×	2.32×	2.31×	2.34×	2.22×	**2.38×**
MG4	26.0 MB	3.47×	3.59×	3.59×	3.62×	3.51×	**3.71×**
CT1	0.50 MB	2.79×	2.49×	2.54×	2.29×	2.70×	**3.19×**
CT2	0.50 MB	3.49×	2.87×	3.11×	2.72×	3.29×	**4.54×**
MG-N	27.3 MB	2.24×	2.32×	2.31×	2.34×	2.23×	**2.38×**
MR1	0.50 MB	2.09×	2.14×	2.10×	2.08×	2.13×	**2.30×**
MR2	2.00 MB	3.28×	3.34×	3.31×	3.31×	3.35×	**3.52×**
MR3	0.50 MB	3.93×	4.09×	3.89×	3.84×	4.33×	**4.51×**
MR4	0.50 MB	4.12×	4.18×	4.09×	4.03×	4.21×	**4.49×**
NM1	0.50 MB	5.15×	5.02×	5.26×	5.28×	5.76×	**6.28×**
RG1	6.86 MB	1.70×	1.70×	1.70×	1.69×	1.63×	**1.72×**
RG2	7.18 MB	4.23×	4.32×	4.28×	4.30×	4.32×	**4.51×**
RG3	5.91 MB	6.08×	6.82×	6.11×	6.12×	6.99×	**7.31×**
SC1	9.71 MB	3.71×	3.70×	3.73×	3.74×	3.85×	**4.73×**
XA1	2.00 MB	5.01×	4.94×	5.04×	5.03×	4.88×	**5.39×**

Table 9: Decompression throughput (MB/s of raw pixel data) across hardware platforms.

Platform	MR	CT	CR	MG1
AWS c7g.metal (ARM64, 64c)	2,282	4,433	8,527	**16,387**
AWS c7g.8xl (ARM64, 32c)	1,524	2,186	4,290	8,901
AWS c7i.8xl (x86, 32c)	1,142	1,208	3,172	5,220
Mac Studio (M2 Max, 12c)	1,054	1,121	2,089	3,666

7 Comparison with HTJ2K (OpenJPH)

We compare against lossless HTJ2K using OpenJPH [33], the leading open-source implementation, via in-process CGO bindings. All measurements are single-threaded on Apple M2 Max.

Compression ratio: MIC leads on images with high dynamic range utilization (where wavelet coefficient overflow penalizes HTJ2K) and on images with large smooth regions (where 16-bit RLE captures long runs efficiently). HTJ2K leads on images with fine spatial structure.

Decompression throughput: HTJ2K decompresses faster than MIC-Go (pure Go, no native optimizations). However, **MIC-4state-C exceeds HTJ2K on 19/21 images single-threaded on ARM64**. Note: on ARM64, MIC-4state-SIMD falls back to scalar C (no AVX2 on Apple Silicon); the speed advantage comes entirely from the four-state ILP decoder design, not SIMD vectorization. The only ARM64 losses are MG1 and MG2 — the two highest-SNR mammography images where HTJ2K's decoder table stays warm in cache.

On AMD64 with `-O3 -march=native`, **MIC-4state-SIMD (AVX2) exceeds HTJ2K on 16/21 images single-threaded** (Table 11). PICS-C-8 (8 parallel strips) **exceeds HTJ2K on all 21 images** on both platforms, with 3–8× higher throughput on large images. At 64 cores, MIC's per-frame parallelism scales to 16 GB/s vs. HTJ2K's single-image architecture.

Complexity: HTJ2K's open-source implementation (OpenJPH [33]) is approximately 20,000 lines of C++ (counted with `cloc`). MIC's equivalent Delta+RLE+FSE pipeline is approximately 500 lines of Go—a 40× complexity difference that matters for maintenance, security auditing, porting, and integration in constrained environments.

Table 11 shows the same comparison on Intel Core Ultra 9 285K (AMD64, 24 P-cores) with `-O3 -march=native`, where MIC-4state-SIMD benefits from AVX2 acceleration and exceeds HTJ2K on 16/21 images single-threaded.

8 Comparison with JPEG-LS (CharLS)

We compare against CharLS [34] via in-process CGO bindings.

Compression ratio: JPEG-LS achieves better ratios on all 21 modalities. The advantage is largest on CT2 (+30%), SC1 (+27%), NM1 (+22%), and RG3 (+20%) where context-adaptive MED prediction captures fine structure.

Decompression throughput: MIC-4state-C is **1.7–5.0× faster on all 21 modalities**. The speed advantage arises from: (1) O(1) table lookups vs. per-pixel parameter updates, (2) four-state ILP breaking the serial ANS bottleneck, and (3) branch-free delta vs. three-way MED conditional.

Summary: JPEG-LS wins on ratio; MIC wins on speed. For clinical workflows where images are compressed once and decompressed many times (archival → viewing), MIC's speed advantage outweighs the modest ratio disadvantage.

Table 10: MIC vs. HTJ2K — in-process, single-threaded, Apple M2 Max (ARM64, `-O3`, no `-march=native`), 21 images. Bold = best ratio or best single-threaded speed.

Image	Raw (MB)	MIC ratio	HTJ2K ratio	MIC-Go (MB/s)	MIC-4s-C (MB/s)	HTJ2K (MB/s)
MR	0.13	2.35×	**2.38**×	144	**348**	265
CT	0.50	**2.24**×	1.77×	191	**356**	307
CR	7.18	3.69×	**3.77**×	296	**524**	367
XR	10.1	**1.74**×	1.67×	308	**533**	334
MG1	9.35	**8.79**×	8.25×	482	683	**810**
MG2	9.35	**8.77**×	8.24×	479	686	**790**
MG3	27.3	**2.24**×	2.22×	308	**531**	338
MG4	26.0	3.47×	**3.51**×	417	**625**	548
CT1	0.50	**2.79**×	2.70×	239	**436**	362
CT2	0.50	**3.49**×	3.29×	238	**439**	375
MG-N	27.3	**2.24**×	2.23×	316	**536**	340
MR1	0.50	2.09×	**2.13**×	278	**521**	325
MR2	2.00	3.28×	**3.35**×	333	**563**	388
MR3	0.50	3.93×	**4.33**×	375	**639**	441
MR4	0.50	4.12×	**4.21**×	316	**571**	406
NM1	0.50	5.15×	**5.76**×	327	**632**	410
RG1	6.86	**1.70**×	1.63×	235	**406**	332
RG2	7.18	4.23×	**4.32**×	367	**590**	443
RG3	5.91	6.08×	**6.99**×	374	**604**	562
SC1	9.71	3.71×	**3.85**×	375	**587**	401
XA1	2.00	**5.01**×	4.88×	331	**576**	419

9 Comparison with Delta+Zstandard: Quantifying the 16-Bit Gap

This section provides the central empirical evidence: that byte-oriented compression systematically underperforms on 16-bit data.

MIC outperforms Delta+zstd on **every modality**, even at zstd's highest compression level (19). The advantage ranges from 10% (MG3) to 22% (MG1). The gap is most pronounced on mammography, where MIC's 16-bit RLE encodes long runs that zstd's byte-oriented LZ77 cannot exploit. The raw+zstd column confirms delta encoding is essential; but even with delta encoding, zstd cannot match MIC's RLE+FSE backend.

10 Parallel Scaling: PICS, Multi-Frame, and WSI

10.1 PICS: Parallel Image Compressed Strips

PICS breaks the serial row dependency in delta prediction by dividing the image into N horizontal strips, each independently compressed/decompressed. Strip boundaries reset the top-neighbor predictor to zero—the only accuracy impact.

The strip adaptation phenomenon: Large images (CR, MG) *improve* compression with more strips because each strip receives a dedicated FSE table that specializes to the strip's local residual distribution. Formally, if each strip S_k has entropy $H(S_k)$, and the image is non-stationary:

$$\sum_k |S_k| \cdot H(S_k) < |S| \cdot H(S) \tag{7}$$

This inequality holds for CR/MG and explains the crossover where strip boundary loss is outweighed by local adaptation gain.

PICS-C-8 achieves up to 3,773 MB/s on MG2 and 3,689 MB/s on MG4 on a 12-core workstation—sub-millisecond decompression. PICS-C-8 exceeds HTJ2K on all 21 images on both ARM64 and AMD64 platforms (see Table 14 and Table 11).

10.2 Multi-Frame Results (MIC2)

For the 69-frame breast tomosynthesis (2457×1890×69, 614 MB raw): independent mode achieves 13.3× compression and temporal mode achieves 12.9×. Independent mode outperforms because tomosynthesis frames differ in X-ray projection angle rather than temporal motion—there is no inter-frame pixel correlation to exploit.

Table 11: MIC vs. HTJ2K — in-process, Intel Core Ultra 9 285K (AMD64, 24 P-cores), `-O3 -march=native`, 21 images. All speeds in MB/s. *Italic* = best single-threaded; **bold** = best PICS-C per row. △ = image too small for PICS-C-8.

Image	Raw (MB)	MIC-Go	MIC-4s-C	MIC-4s-SIMD	HTJ2K	PICS-C-2	PICS-C-4	PICS-C-8
MR	0.13	251	501	*708*	570	**340**	301	124 △
CT	0.50	231	403	487	*544*	534	**676**	339
CR	7.18	324	599	*744*	708	908	1,738	**2,435**
XR	10.1	363	601	*803*	570	982	1,714	**1,994**
MG1	9.35	617	789	1,119	*1,235*	1,511	1,872	**2,514**
MG2	9.35	562	800	1,166	*1,297*	1,397	**2,244**	2,085
MG3	27.3	357	633	*669*	644	1,078	1,823	**2,538**
MG4	26.0	523	743	773	*916*	1,612	2,124	**2,707**
CT1	0.50	322	520	*676*	657	636	**857**	423
CT2	0.50	293	514	636	627	645	**847**	687
MG-N	27.3	368	635	669	643	1,128	1,872	**2,191**
MR1	0.50	356	609	*766*	654	645	**945**	366
MR2	2.00	352	658	*975*	749	963	**1,121**	793
MR3	0.50	450	728	*809*	802	786	**920**	705
MR4	0.50	390	596	*861*	660	528	493	**701**
NM1	0.50	367	*717*	668	627	663	**783**	405
RG1	6.86	302	497	593	557	787	1,248	**1,494**
RG2	7.18	433	676	*861*	823	1,160	1,841	**1,912**
RG3	5.91	460	706	858	*894*	1,309	**1,969**	1,613
SC1	9.71	464	695	*888*	728	1,169	1,819	**1,889**
XA1	2.00	413	693	*836*	797	1,035	1,173	**1,513**

10.3 WSI Compression (MIC3)

MIC3 is a tiled container for whole slide pathology images with YCoCg-R reversible color transform, 256×256 tiles with O(1) random access, multi-resolution pyramid levels, and a constant-plane optimization compressing background tiles to 15–17 bytes. Representative ratios: white background (1,946×), dense H&E tissue (4.4×), gradient tiles (5.4×).

10.4 Single-Frame RGB: Ultrasound and Visible Light (MICR)

Single-frame RGB modalities use `CompressRGB`/`DecompressRGB` applying YCoCg-R + Delta+RLE+FSE to the full image without tile boundaries. The compressed blob is wrapped in a minimal MICR container (4-byte magic + width + height) for browser delivery. Using the tiled MIC3 for non-tiled images causes a 30–45% ratio loss due to predictor restarts at tile boundaries.

Compression ratios on NEMA compsamples: US1 (640×480) achieves 6.24×; VL1–VL3 (756×486) achieve 3.2–3.5×; high-detail VL4–VL6 achieve 1.56–1.93×.

11 Implementation Notes

11.1 Go Assembly Optimizations

MIC includes platform-specific Go assembly for amd64 and arm64 with pure-Go fallbacks:

- **Four-state FSE decode kernel**: BMI2 `SHLXQ`/`SHRXQ` on amd64; `LSLV`/`LSRV` on arm64.
- **Interleaved histogram** (`countSimpleU16Asm`): 4-way unrolled loop distributing even/odd pixels into separate count arrays to avoid store-to-load forwarding stalls.
- **AVX2 wavelet kernels**: `wt53PredictAVX2`/`wt53UpdateAVX2` processing 8 int32 values per cycle for the blocked column pass.

11.2 Branch-Free Delta Decompression

The delta decompressor uses four separate loops for corner, first-row, first-column, and interior pixels. The interior loop handles $> (w-1)(h-1)$ out of wh pixels without any per-pixel boundary checks, eliminating branch mispredictions.

Table 12: MIC vs. JPEG-LS — in-process, single-threaded, Apple M2 Max, 21 images. Speedup = MIC-4s-C / JPEG-LS.

Image	Raw (MB)	MIC ratio	JPEG-LS ratio	MIC-Go	MIC-4s-C	JPEG-LS	Speedup
MR	0.13	2.35×	**2.52**×	144	**348**	102	3.4×
CT	0.50	2.24×	**2.68**×	191	**356**	137	2.6×
CR	7.18	3.69×	**3.96**×	296	**524**	153	3.4×
XR	10.1	1.74×	**1.76**×	308	**533**	108	4.9×
MG1	9.35	8.79×	**8.91**×	482	**683**	409	1.7×
MG2	9.35	8.77×	**8.90**×	479	**686**	416	1.6×
MG3	27.3	2.24×	**2.38**×	308	**531**	153	3.5×
MG4	26.0	3.47×	**3.71**×	417	**625**	184	3.4×
CT1	0.50	2.79×	**3.19**×	239	**436**	182	2.4×
CT2	0.50	3.49×	**4.54**×	238	**439**	175	2.5×
MG-N	27.3	2.24×	**2.38**×	316	**536**	153	3.5×
MR1	0.50	2.09×	**2.30**×	278	**521**	116	4.5×
MR2	2.00	3.28×	**3.52**×	333	**563**	172	3.3×
MR3	0.50	3.93×	**4.51**×	375	**639**	236	2.7×
MR4	0.50	4.12×	**4.49**×	316	**571**	197	2.9×
NM1	0.50	5.15×	**6.28**×	327	**632**	210	3.0×
RG1	6.86	**1.70**×	**1.72**×	235	**406**	104	3.9×
RG2	7.18	4.23×	**4.51**×	367	**590**	193	3.1×
RG3	5.91	6.08×	**7.31**×	374	**604**	246	2.5×
SC1	9.71	3.71×	**4.73**×	375	**587**	229	2.6×
XA1	2.00	**5.01**×	**5.39**×	331	**576**	204	2.8×

Table 13: MIC vs. Delta+Zstandard at varying compression levels and raw+Zstandard. MIC (adaptive tableLog) vs. d+zstd at levels 1, 3, 19.

Image	MIC	d+z-1	d+z-3	d+z-19	z-3
MR	**2.35**×	1.75×	1.82×	1.95×	1.65×
CT	**2.24**×	1.71×	1.89×	2.03×	1.79×
CR	**3.63**×	2.70×	2.95×	3.27×	2.05×
XR	**1.74**×	1.43×	1.43×	1.43×	1.32×
MG1	**8.57**×	6.19×	6.37×	7.07×	5.77×
MG2	**8.55**×	6.18×	6.36×	7.04×	5.75×
MG3	**2.29**×	1.71×	1.89×	2.09×	1.50×
MG4	**3.47**×	2.80×	2.87×	2.99×	2.57×

11.3 RLE Fast Path

Same-value runs (the most common pattern after delta coding) are fast-pathed to return the cached value without touching the input slice, requiring only a counter decrement per output symbol.

11.4 Browser Decoders — Web Distribution Without a Server

A practical advantage MIC holds over all DICOM-standard codecs is **browser-native decoding**. HTJ2K has no production-ready browser decoder [35]; JPEG-LS has none either. To view a JPEG 2000 image in a web browser today, the server must transcode it.

MIC ships two browser decoder implementations: a pure JavaScript ES module (∼20 KB, zero dependencies) and a Go WebAssembly build (∼2.5 MB). The JavaScript decoder handles FSE's 64-bit reverse bit reader using `BigInt` with explicit uint64 masking, and works in any browser since 2020 (Chrome 67+, Firefox 68+, Safari 14+).

Parallel decoding via PICS + `worker_threads`: PICS files encode independent strips, enabling parallel browser decoding without `SharedArrayBuffer` synchronization.

MG1 (9.35 MB) decompresses in **19.4 ms at 483 MB/s** using 12 browser workers—well into real-time territory for diagnostic viewing. A web-based DICOM viewer can fetch a `.mic` file directly from object storage and decode it client-side: zero server-side compute, zero transcoding latency, works offline and in service workers.

12 Discussion

12.1 The Cost-of-Complexity Frontier

JPEG-LS trades 58% of decompression speed for 10% more compression—unfavorable for clinical systems that decompress 10× more often than they

Table 14: Single-image decompression throughput (MB/s), Apple M2 Max (12-core ARM64), C variants `-O3`. PICS-C uses C pthreads. △ = MR too small for PICS-C-8; PICS-C-4 is best.

Image	MIC-Go	MIC-4s-C	PICS-C-2	PICS-C-4	PICS-C-8
MR	144	348	530	**710**	482 △
CT	191	356	524	955	**1,092**
CR	296	524	867	1,635	**2,661**
XR	308	533	874	1,666	**3,025**
MG1	482	683	1,205	2,112	**3,656**
MG2	479	686	1,225	2,120	**3,773**
MG3	308	531	864	1,673	**3,117**
MG4	417	625	1,093	2,004	**3,689**
CT1	239	436	686	1,013	**1,183**
CT2	238	439	676	1,041	**1,189**
MG-N	316	536	883	1,711	**3,175**
MR1	278	521	751	1,207	**1,402**
MR2	333	563	913	1,552	**2,466**
MR3	375	639	908	1,430	**1,614**
MR4	316	571	818	1,341	**1,558**
NM1	327	632	888	1,400	**1,679**
RG1	235	406	602	1,128	**2,017**
RG2	367	590	986	1,803	**3,194**
RG3	374	604	1,035	1,944	**3,302**
SC1	375	587	1,017	1,861	**3,279**
XA1	331	576	928	1,583	**2,493**

Table 15: PICS parallel JavaScript decoder, `worker_threads`, Apple M2 Max.

Image	Strips	Workers	ms	MB/s	Speedup
MR 256×256	4	8	1.3	94	2.57×
CT 512×512	4	8	5.0	101	2.95×
CR 1760×2140	8	12	31.7	227	5.53×
MG1 1996×2457	8	12	19.4	**483**	4.36×

compress. MIC's 4-state-C variant achieves 4× the throughput of JPEG-LS at 91% of its ratio. HTJ2K's ∼20,000-line implementation is ∼40× larger than MIC's Delta+RLE+FSE pipeline and has no practical browser decoder [35].

12.2 Pipeline Selection Heuristic

12.3 Limitations

- No lossy compression, progressive decoding, or region-of-interest coding for greyscale images. The wavelet pipeline provides a natural foundation for a future lossy mode.
- **Encoding speed not reported**: This paper focuses on decompression throughput, consistent with the write-once, read-many PACS archival deployment model.
- **Test dataset**: 21 images across 10 modalities from three public de-identified repositories. Large-scale benchmarks from TCIA with inter-institution variability would further strengthen generality claims.
- **Temporal prediction** underperforms on the one available clinical dataset; evaluation on cardiac cine MRI and fluoroscopy is needed.
- **WSI results** are on synthetic tiles; real whole-slide pathology benchmarks are needed.
- **Benchmark confidence intervals**: run-to-run variance < 2% on Apple M2 Max and < 5% on AWS; formal confidence intervals deferred to future work.
- **JPEG-XL** [6]: designed for 8-bit natural images with no 16-bit DICOM pathway; not compared.

Table 16: Pareto frontier: compression ratio vs. decompression throughput (approximate geometric means), with implementation complexity and browser deployability.

Codec	Ratio	Decomp.	LOC	Browser
HTJ2K	3.12×	310 MB/s	∼500	**Yes (20 KB)**
Delta+RLE+FSE (Go)	3.12×	530 MB/s	∼1,500	**Yes**
Delta+RLE+FSE (4s-C)	3.28×	780 MB/s	∼2,000	Possible
Wavelet V2 SIMD	3.15×	460 MB/s	∼20,000	No
HTJ2K (OpenJPH)	3.44×	130 MB/s	∼5,000	No
JPEG-LS (CharLS)	2.72×	∼300 MB/s	N/A	Partial
Delta+zstd-19				

Table 17: Pipeline selection decision framework.

Condition	Recommended Pipeline
Eff. bit depth ≤ 12, speed priority	Delta+RLE+FSE (4-state)
Eff. bit depth ≤ 12, ratio priority	Wavelet V2 SIMD
Full 16-bit dynamic range (CT)	Delta+RLE+FSE
Image ≥ 0.5 MB, multi-core avail.	Add PICS (2–8 strips)
Multi-frame sequence	MIC2 independent mode
Single-frame RGB (US, VL)	MICR (full-image, no tiling)
RGB pathology / WSI	MIC3 (tiled)
Lossy/progressive needed	Use HTJ2K

12.4 Pathway to Clinical Impact

MIC could be registered as a DICOM Private Transfer Syntax, allowing PACS vendors to adopt it without modifying the DICOM standard. For whole slide imaging, DICOM Supplement 145 [29] defines the WSI IOD; MIC3's tiled format aligns with this architecture. Herrmann et al. [28] describe practical challenges of DICOM for digital pathology—MIC3's tile-level random access and pyramid support address these requirements directly.

13 Conclusion

We have presented MIC, a lossless medical image codec built on a simple observation: the compression ecosystem has a 30-year blind spot for 16-bit data. By extending FSE to 65,535 symbols and pairing it with a 16-bit-native RLE stage, MIC outperforms byte-oriented Zstandard by 10–22% on all tested modalities. The four-state interleaved ANS decoder demonstrates that entropy coder design should target instruction-level parallelism width, achieving 66–142% FSE speedup with no ratio penalty.

The simplicity thesis is validated empirically: a three-stage Delta+RLE+FSE pipeline (∼500 lines of code) matches or beats JPEG 2000's wavelet+EBCOT on 7/8 medical modalities, at dramatically higher throughput. When maximum ratio is needed, a five-level wavelet alternative with SIMD acceleration exceeds both the delta pipeline and HTJ2K.

Key results across 21 clinical DICOM images spanning 10 modalities:

- **Compression ratios**: 1.7×–8.9× (greyscale), 3–5× (RGB WSI tissue), 1.56×–6.24× (single-frame RGB US/VL)
- **Decompression throughput**: up to 16 GB/s (64-core ARM64), geometric mean ≈7.5 GB/s
- **vs. HTJ2K**: MIC-4state-C exceeds HTJ2K on 19/21 images single-threaded (ARM64; speed gain from 4-state ILP — no AVX2 on Apple Silicon); MIC-4state-SIMD (AVX2) exceeds HTJ2K on 16/21 images single-threaded (Intel AMD64, `-march=native`); PICS-C-8 exceeds HTJ2K on **all 21 images**; MIC pipeline is ∼40× simpler
- **vs. JPEG-LS**: 1.7–5.0× faster decompression; 1–30% lower ratio
- **vs. Delta+Zstandard**: 10–22% better compression on all modalities
- **Browser decoder**: ∼20 KB pure JS decoder; PICS + `worker_threads` achieves 483 MB/s (MG1) with no server-side transcoding
- **Portability**: Four implementations span the full deployment spectrum—Pure Go (no CGO, single binary) → C/pthreads+SIMD → JavaScript ES module (∼20 KB, zero npm dependencies) → Go WebAssembly. No native libraries are required for browser-native decoding; HTJ2K and JPEG-LS have no practical browser decoder.
- **Compact**: The entire Delta+RLE+FSE pipeline is ∼500 lines of Go code—∼40× less than HTJ2K's ∼20,000-line implementation—yet matches or exceeds HTJ2K on 7/8 modalities for both ratio and decompression speed.

MIC is open-source and available at https://github.com/pappuks/medical-image-codec.

References

[1] NEMA, "Digital Imaging and Communications in Medicine (DICOM)," https://www.dicomstandard.org/, 2024.

[2] D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Springer, 2002.

[3] D. S. Taubman, "High throughput JPEG 2000 (HTJ2K): Algorithm, performance evaluation, and potential," *Proc. SPIE*, vol. 11137, 2019.

[4] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Trans. Image Process.*, vol. 9, no. 8, pp. 1309–1324, 2000.

[5] ISO/IEC 14495-2:2003, "Information technology — Lossless and near-lossless compression of continuous-tone still images: Extensions — Part 2," 2003.

[6] J. Alakuijala *et al.*, "JPEG XL next-generation image compression architecture and coding tools," *Proc. SPIE 11137*, 111370K, Sep. 2019.

[7] J. Duda, "Asymmetric numeral systems," arXiv:0902.0271, 2009.

[8] J. Duda, "Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding," arXiv:1311.2540, Nov. 2013.

[9] Y. Collet, "Finite State Entropy — A new breed of entropy coder," https://github.com/Cyan4973/FiniteStateEntropy, 2013.

[10] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, IETF, Feb. 2021.

[11] E. S. Schwartz and B. Kallick, "Generating a canonical prefix encoding," *Commun. ACM*, vol. 7, no. 3, pp. 166–169, 1964.

[12] S. Mahapatra and K. Singh, "An FPGA-based implementation of multi-alphabet arithmetic coding," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 54, no. 8, pp. 1678–1686, Aug. 2007.

[13] D. Kosolobov, "Efficiency of ANS Entropy Encoders," arXiv:2201.02514, Jan. 2022.

[14] R. Bamler, "Understanding Entropy Coding With Asymmetric Numeral Systems (ANS): a Statistician's Perspective," arXiv:2201.01741, Jan. 2022.

[15] F. Giesen, "Interleaved entropy coders," arXiv:1402.3392, Feb. 2014.

[16] F. Giesen, "ryg_rans: Public domain rANS encoder/decoder," https://github.com/rygorous/ryg_rans, 2014.

[17] F. Lin, K. Arunruangsirilert, H. Sun, and J. Katto, "Recoil: Parallel rANS Decoding with Decoder-Adaptive Scalability," *Proc. 52nd Int. Conf. Parallel Processing (ICPP '23)*, pp. 31–40, 2023.

[18] A. Weissenberger and B. Schmidt, "Massively Parallel ANS Decoding on GPUs," *Proc. 48th Int. Conf. Parallel Processing (ICPP '19)*, Article 100, 2019.

[19] X. Wu and N. Memon, "Context-based, adaptive, lossless image coding," *IEEE Trans. Commun.*, vol. 45, no. 4, pp. 437–444, Apr. 1997.

[20] J. Sneyers and P. Wuille, "FLIF: Free Lossless Image Format based on MANIAC Compression," *Proc. IEEE ICIP*, Phoenix, AZ, Sep. 2016.

[21] D. Le Gall and A. Tabatabai, "Sub-band coding of digital images using symmetric short kernel filters and arithmetic coding techniques," *Proc. IEEE ICASSP*, pp. 761–764, 1988.

[22] W. Sweldens, "The Lifting Scheme: A custom-design construction of biorthogonal wavelets," *Appl. Comput. Harmon. Anal.*, vol. 3, no. 2, pp. 186–200, 1996.

[23] I. Daubechies and W. Sweldens, "Factoring wavelet transforms into lifting steps," *J. Fourier Anal. Appl.*, vol. 4, no. 3, pp. 247–269, 1998.

[24] A. R. Calderbank, I. Daubechies, W. Sweldens, and B.-L. Yeo, "Wavelet transforms that map integers to integers," *Appl. Comput. Harmon. Anal.*, vol. 5, no. 3, pp. 332–369, 1998.

[25] H. S. Malvar and G. J. Sullivan, "YCoCg-R: A color space with RGB reversibility and low dynamic range," JVT Doc. JVT-I014, 2003.

[26] F. Liu, M. Hernandez-Cabronero, V. Sanchez, M. W. Marcellin, and A. Bilgin, "The Current Role of Image Compression Standards in Medical Imaging," *Information*, vol. 8, no. 4, p. 131, Oct. 2017.

[27] D. A. Clunie, "Lossless Compression of Grayscale Medical Images: Effectiveness of Traditional and State-of-the-Art Approaches," *Proc. SPIE Medical Imaging*, 2000.

[28] M. D. Herrmann, D. A. Clunie, A. Fedorov *et al.*, "Implementing the DICOM Standard for Digital Pathology," *J. Pathol. Inform.*, vol. 9, no. 1, p. 37, 2018.

[29] DICOM Standards Committee, "Supplement 145: Whole Slide Microscopic Image IOD and SOP Classes," DICOM Standard, 2010.

[30] D. Taubman, A. Naman, M. Smith *et al.*, "High Throughput JPEG 2000 for Video Content Production and Delivery Over IP Networks," *Frontiers in Signal Processing*, vol. 2, Article 885644, Apr. 2022.

[31] D. Taubman, A. Naman, and R. Mathew, "High Throughput Block Coding in the HTJ2K Compression Standard," *Proc. IEEE ICIP*, 2019.

[32] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[33] A. N. Aous, "OpenJPH: An open-source implementation of High-Throughput JPEG 2000," https://github.com/aous72/OpenJPH, 2024.

[34] Team CharLS, "CharLS: A C/C++ JPEG-LS library implementation," https://github.com/team-charls/charls, 2024.

[35] As of the submission date of this paper, no production-ready WebAssembly or JavaScript decoder exists for High-Throughput JPEG 2000.

The OpenJPH repository does not provide a WASM build or npm package. Readers are encouraged to verify current availability at https://github.com/aous72/OpenJPH/issues.

[36] N. Tindall *et al.*, "fzstd: Pure JavaScript Zstandard decompressor," https://github.com/101arrowz/fzstd, 2024.